INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 12 LECTURE 2

VIRTUAL FILE SYSTEM

WISH YOU ALL THE BEST

LECTURE OUTLINE

- Aims, Structure, and Implementation of Linux's Virtual Filesystem
- Three MAIN Standard Unix file types
  - Regular files
  - Directories
  - Symbolic links

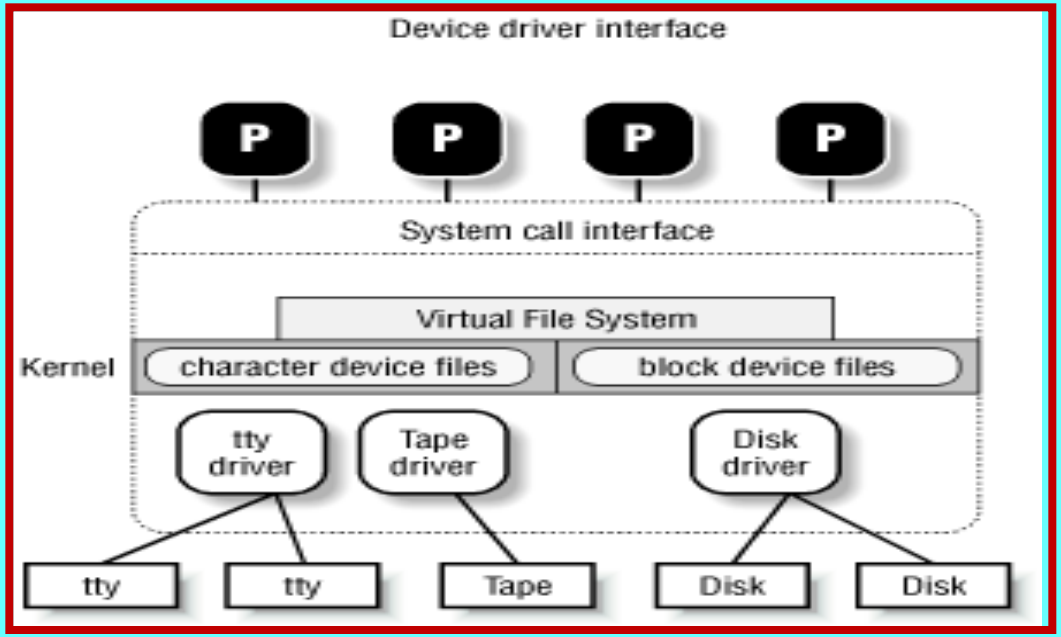
Virtual File System

- **Virtual File System (VFS) - One of Linux's keys to success is its ability to coexist comfortably with other systems.**
- **You can transparently mount disks or partitions that host file formats used by Windows , other Unix systems, or even systems with tiny market shares like the Amiga.**
- **Linux manages to support multiple filesystem types in the same way other Unix variants do, through a concept called the Virtual File System.**
- **The idea behind the Virtual Filesystem is to put a wide range of information in the kernel to represent many different types of filesystems ; there is a field or function to support each operation provided by all real filesystems supported by Linux.**
- **For each read, write, or other function called, the kernel substitutes the actual function that supports a native Linux filesystem, the NTFS filesystem, or whatever other filesystem the file is on.**

Virtual File System

❑ Role of the Virtual Filesystem (VFS)

- The Virtual Filesystem (also known as Virtual Filesystem Switch or VFS) is a kernel software layer that handles all system calls related to a standard Unix filesystem.
- Its main strength is providing a common interface to several kinds of filesystems.
- The first Virtual Filesystem was included in Sun Microsystems's SunOS in 1986. Since then, most Unix filesystems include a VFS. Linux's VFS, however, supports the widest range of filesystems.



Virtual File System

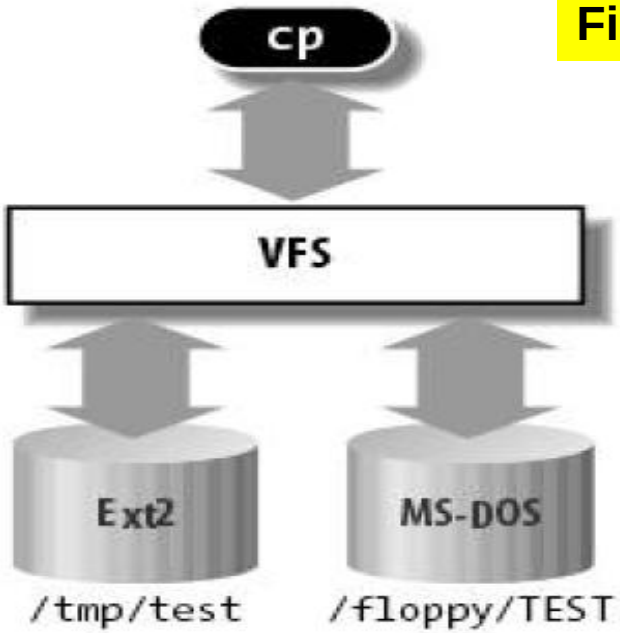
❑ Role of the Virtual Filesystem (VFS)

- For instance, let's assume that a user issues the shell command:

  $ cp /floppy/TEST /tmp/test

  where /floppy is the mount point of an MS-DOS diskette and /tmp is a normal Second Extended Filesystem (Ext2) directory.
- The VFS is an abstraction layer between the application program and the filesystem implementations
- Therefore, the cp program is not required to know the filesystem types of /floppy/TEST and /tmp/test.

Figure - VFS role in a simple file copy operation



(a)

```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test",
       O_WRONLY|O_CREAT|O_TRUNC, 0600);
do {
    i = read(inf, buf, 4096);
    write(outf, buf, i);
} while (i);
close(outf);
close(inf);
```

(b)

cp interacts with the VFS by means of generic system calls, the code executed by cp is shown in Figure

Virtual File System

- **Filesystems supported by the VFS may be grouped into three main classes:**

➢ Disk-based filesystems

- **These manage memory space available in a local disk or in some other device that emulates a disk (such as a USB flash drive).**

➢ Network filesystems

- **These allow easy access to files included in filesystems belonging to other networked computers.**
- **Some well-known network filesystems supported by the VFS are NFS , Coda , AFS (Andrew filesystem), CIFS (Common Internet File System, used in Microsoft Windows), and NCP (Novell's NetWare Core Protocol).**

❑ Special filesystems

- **These do not manage disk space, either locally or remotely. The /proc filesystem is a typical example of a special filesystem**

Virtual File System

❑ Disk-based filesystems

Some of the well-known disk-based filesystems supported by the VFS are:

- **Filesystems for Linux such as the widely used Extended Filesystem (Ext2, Ex3, Ex4), and the Reiser Filesystems (ReiserFS)**
- **Filesystems for Unix variants such as sysv filesystem (System V, Coherent, Xenix), UFS (BSD , Solaris , NEXTSTEP), MINIX filesystem, and VERITAS VxFS (SCO UnixWare)**
- **Microsoft filesystems such as MS-DOS, VFAT (Windows 95 and later releases), and NTFS (Windows NT 4 and later releases)**
- **ISO9660 CD-ROM filesystem (formerly High Sierra Filesystem) and Universal Disk Format (UDF) DVD filesystem**
- **Other proprietary filesystems such as IBM's OS/2 (HPFS), Apple's Macintosh (HFS), Amiga's Fast Filesystem (AFFS), and Acorn Disk Filing System (ADFS)**
- **Additional journaling filesystems originating in systems other than Linux such as IBM's JFS and SGI's XFS**

Virtual File System

❑ Common File Model

- The key idea behind the VFS consists of introducing a common file model capable of representing all supported filesystems.
- This model strictly mirrors the file model provided by the traditional Unix filesystem.
- Linux wants to run its native filesystem with minimum overhead.
- However, each specific filesystem implementation must translate its physical organization into the VFS's common file model.

- For instance, in the common file model, each directory is regarded as a file, which contains a list of files and other directories.
- However, several non-Unix disk-based filesystems use a File Allocation Table (FAT), which stores the position of each file in the directory tree.
- In these filesystems, directories are not files.
- To stick to the VFS's common file model, the Linux implementations of such FAT-based filesystems must be able to construct on the fly, when needed, the files corresponding to the directories.
- Such files exist only as objects in kernel memory.

Virtual File System

❑ Common File Model

- Common file model can be thought as object-oriented, where an object is a software construct that defines both a data structure and the methods that operate on it.
- For reasons of efficiency, Linux is not coded in an object-oriented language such as C++.
- Objects are therefore implemented as plain C data structures with some fields pointing to functions that correspond to the object's methods.

Virtual File System

❑ Common File Model

➢ The common file model consists of the following object types:

❖ **The superblock object**

Stores information concerning a mounted filesystem. For disk-based filesystems, this object usually corresponds to a filesystem control block stored on disk.

❖ **The inode object**

Stores general information about a specific file. For disk-based filesystems, this object usually corresponds to a file control block stored on disk. Each inode object is associated with an inode number, which uniquely identifies the file within the filesystem.

❖ **The file object**

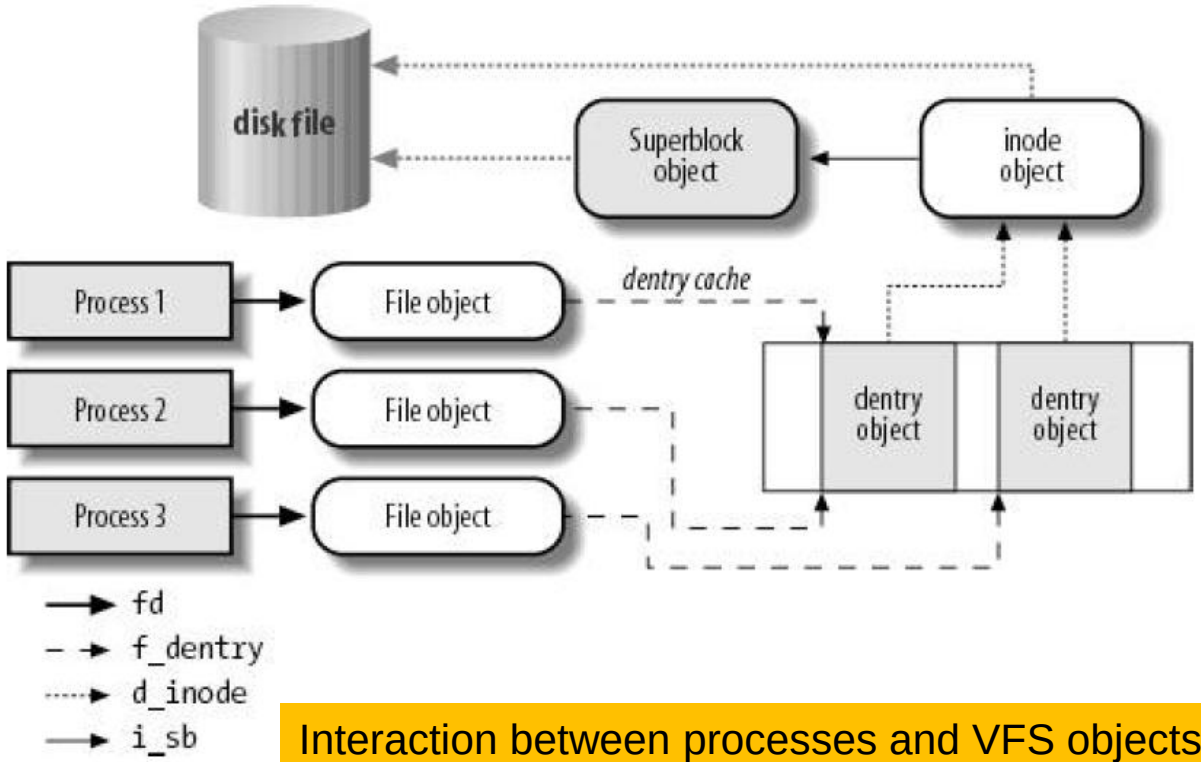
Stores information about the interaction between an open file and a process. This information exists only in kernel memory during the period when a process has the file open.

❖ **The dentry object**

Stores information about the linking of a directory entry (that is, a particular name of the file) with the corresponding file. Each disk-based filesystem stores this information in its own particular way on disk.

Virtual File System

- **A simple example to show how processes interact with files**
  - **Three different processes have opened the same file, two of them using the same hard link.**
  - **In this case, each of the three processes uses its own file object, while only two dentry objects are required one for each hard link.**
  - **Both dentry objects refer to the same inode object, which identifies the superblock object and, together with the latter, the common disk file.**



Interaction between processes and VFS objects

- **Besides providing a common interface to all filesystem implementations, the VFS has another important role related to system performance.**
- **The most recently used dentry objects are contained in a disk cache named the dentry cache, which speeds up the translation from a file pathname to the inode of the last pathname component.**

Virtual File System

❑ System Calls Handled by the VFS

Table- Some system calls handled by the VFS

System call name	Description
mount( ) umount( ) umount2( )	Mount/unmount filesystems
sysfs( )	Get filesystem information
statfs( ) fstatfs( ) statfs64( ) fstatfs64( ) ustat( )	Get filesystem statistics
chroot( ) pivot_root( )	Change root directory
chdir( ) fchdir( ) getcwd( )	Manipulate current directory
mkdir( ) rmdir( )	Create and destroy directories
getdents( ) getdents64( ) readdir( ) link( ) unlink( ) rename( ) lookup_dcookie( )	Manipulate directory entries
readlink( ) symlink( )	Manipulate soft links
chown( ) fchown( ) lchown( ) chown16( ) fchown16( ) lchown16( )	Modify file owner
chmod( ) fchmod( ) utime( )	Modify file attributes
stat( ) fstat( ) lstat( ) access( ) oldstat( ) oldfstat( ) oldlstat( ) stat64( ) lstat64( ) fstat64( )	Read file status
open( ) close( ) creat( ) umask( )	Open, close, and create files
dup( ) dup2( ) fcntl( ) fcntl64( )	Manipulate file descriptors
select( ) poll( )	Wait for events on a set of file descriptors

Virtual File System

❑ System Calls Handled by the VFS

Table- Some system calls handled by the VFS

System call name	Description
truncate( ) ftruncate( ) truncate64( ) ftruncate64( )	Change file size
lseek( ) _llseek( )	Change file pointer
read( ) write( ) readv( ) writev( ) sendfile( ) sendfile64( ) readahead( )	Carry out file I/O operations
io_setup( ) io_submit( ) io_getevents( ) io_cancel( ) io_destroy( )	Asynchronous I/O (allows multiple outstanding read and write requests)
pread64( ) pwrite64( )	Seek file and access it
mmap( ) mmap2( ) munmap( ) madvise( ) mincore( ) remap_file_pages( )	Handle file memory mapping
fdatasync( ) fsync( ) sync( ) msync( )	Synchronize file data
flock( )	Manipulate file lock
setxattr( ) lsetxattr( ) fsetxattr( ) getxattr( ) lgetxattr( ) fgetxattr( ) listxattr( ) llistxattr( ) flistxattr( ) removexattr( ) lremovexattr( ) fremovexattr( )	Manipulate file extended attributes

- VFS is a layer between application programs and specific filesystems.

Virtual File System

❑ VFS Data Structures

- Each VFS object is stored in a suitable data structure, which includes both the object attributes and a pointer to a table of object methods.
- The kernel may dynamically modify the methods of the object and, hence, it may install specialized behavior for the object.

▪ Superblock Objects

A superblock object consists of a super_block structure whose fields are described in Table

Table- The fields of the superblock object

Type	Field	Description
struct list_head	s_list	Pointers for superblock list
dev_t	s_dev	Device identifier
unsigned long	s_blocksize	Block size in bytes
unsigned long	s_old_blocksize	Block size in bytes as reported by the underlying block device driver
unsigned char	s_blocksize_bits	Block size in number of bits
unsigned char	s_dirt	Modified (dirty) flag
unsigned long	long s_maxbytes	Maximum size of the files
Struct file_system_type *	s_type	Filesystem type
Struct super_operations *	s_op	Superblock methods
struct dquot_operations *	dq_op	Disk quota handling methods
struct quotactl_ops *	s_qcop	Disk quota administration methods

Virtual File System

❑ VFS Data Structures

- **Superblock Objects**

Table- The fields of the superblock object

Type	Field	Description
struct export_operations*	s_export_op	Export operations used by network filesystems
unsigned long	s_flags	Mount flags
unsigned long	s_magic	Filesystem magic number
struct dentry *	s_root	Dentry object of the filesystem's root directory
struct rw_semaphore	s_umount	Semaphore used for unmounting
struct semaphore	s_lock	Superblock semaphore
int	s_count	Reference counter
int	s_syncing	Flag indicating that inodes of the superblock are being synchronized
int	s_need_sync_fs	Flag used when synchronizing the superblock's mounted filesystem
atomic_t	s_active	Secondary reference counter
void *	s_security	Pointer to superblock security structure
struct xattr_handler **	s_xattr	Pointer to superblock extended attribute structure
struct list_head	s_inodes	List of all inodes
struct list_head	s_dirty	List of modified inodes

Virtual File System

❑ VFS Data Structures

- **Superblock Objects**

Table- The fields of the superblock object

Type	Field	Description
struct list_head	s_io	List of inodes waiting to be written to disk
struct hlist_head	s_anon	List of anonymous dentries for handling remote network filesystems
struct list_head	s_files	List of file objects
struct block_device *	s_bdev	Pointer to the block device driver descriptor
struct list_head	s_instances	Pointers for a list of superblock objects of a given filesystem type
struct quota_info	s_dquot	Descriptor for disk quota
int	s_frozen	Flag used when freezing the filesystem (forcing it to a consistent state)
wait_queue_head_t	s_wait_unfrozen	Wait queue where processes sleep until the filesystem is unfrozen
char[]	s_id	Name of the block device containing the superblock
void *	s_fs_info	Pointer to superblock information of a specific filesystem
struct semaphore	s_vfs_rename_sem	Semaphore used by VFS when renaming files across directories
u32	s_time_gran	Timestamp's granularity (in nanoseconds)

Virtual File System

❑ Superblock Operations

- The methods associated with a superblock are called superblock operations.
- They are described by the super_operations structure whose address is included in the s_op field.
- Each specific filesystem can define its own superblock operations.
- When the VFS needs to invoke one of them, say read_inode( ), it executes the following:

  `sb->s_op->read_inode(inode);`

  where sb stores the address of the superblock object involved.
- The read_inode field of the super_operations table contains the address of the suitable function, which is therefore directly invoked.

Virtual File System

❑ Superblock Operations

The superblock operations, which implement higher-level operations like deleting files or mounting disks are listed below in the order in which they appear in the super_operations table :

alloc_inode(sb)
Allocates space for an inode object, including the space required for filesystem-specific data.

destroy_inode(inode)
Destroys an inode object, including the filesystem-specific data.

read_inode(inode)
Fills the fields of the inode object passed as the parameter with the data on disk; the i_ino field of the inode object identifies the specific filesystem inode on the disk to be read.

dirty_inode(inode)
Invoked when the inode is marked as modified (dirty). Used by filesystems such as ReiserFS and Ext3/4 to update the filesystem journal on disk.

Virtual File System

❑ Superblock Operations

write_inode(inode, flag)
Updates a filesystem inode with the contents of the inode object passed as the parameter; the i_ino field of the inode object identifies the filesystem inode on disk that is concerned. The flag parameter indicates whether the I/O operation should be synchronous.

put_inode(inode)
Invoked when the inode is released its reference counter is decreased to perform filesystem-specific operations.

drop_inode(inode)
Invoked when the inode is about to be destroyed that is, when the last user releases the inode; filesystems that implement this method usually make use of generic_drop_inode( ).

generic_drop_inode( )
This function removes every reference to the inode from the VFS data structures and, if the inode no longer appears in any directory, invokes the delete_inode superblock method to delete the inode from the filesystem.

Virtual File System

❑ Superblock Operations

delete_inode(inode)
Invoked when the inode must be destroyed. Deletes the VFS inode in memory and the file data and metadata on disk.

put_super(super)
Releases the superblock object passed as the parameter (because the corresponding filesystem is unmounted).

write_super(super)
Updates a filesystem superblock with the contents of the object indicated.

sync_fs(sb, wait)
Invoked when flushing the filesystem to update filesystem-specific data structures on disk (used by journaling filesystems).

write_super_lockfs(super)
Blocks changes to the filesystem and updates the superblock with the contents of the object indicated. This method is invoked when the filesystem is frozen, for instance by the Logical Volume Manager (LVM) driver.

Virtual File System

❑ Superblock Operations

unlockfs(super)
Undoes the block of filesystem updates achieved by the write_super_lockfs superblock method.

statfs(super, buf)
Returns statistics on a filesystem by filling the buf buffer.

remount_fs(super, flags, data)
Remounts the filesystem with new options (invoked when a mount option must be changed).

clear_inode(inode)
Invoked when a disk inode is being destroyed to perform filesystem-specific operations.

umount_begin(super)
Aborts a mount operation because the corresponding unmount operation has been started (used only by network filesystems).

show_options(seq_file, vfsmount)
Used to display the filesystem-specific options

Virtual File System

❑ Superblock Operations

quota_read(super, type, data, size, offset)
Used by the quota system to read data from the file that specifies the limits for this filesystem.The quota system defines for each user and/or group limits on the amount of space that can be used on a given filesystem (quotactl() system call.)
quota_write(super, type, data, size, offset)
Used by the quota system to write data into the file that specifies the limits for this filesystem.
put_super(super)
Releases the superblock object passed as the parameter (because the corresponding filesystem is unmounted).
write_super(super)
Updates a filesystem superblock with the contents of the object indicated.
sync_fs(sb, wait)
Invoked when flushing the filesystem to update filesystem-specific data structures on disk (used by journaling filesystems).
write_super_lockfs(super)
Blocks changes to the filesystem and updates the superblock with the contents of the object indicated. This method is invoked when the filesystem is frozen, for instance by the Logical Volume Manager (LVM) driver.
unlockfs(super)
Undoes the block of filesystem updates achieved by the write_super_lockfs superblock method.

Virtual File System

❑ Inode Object

- All information needed by the filesystem to handle a file is included in a data structure called an inode.
- A filename is a casually assigned label that can be changed, but the inode is unique to the file and remains the same as long as the file exists.
- An inode object in memory consists of an inode structure whose fields are described in Table

Table - Fields of the inode object

Type	Field	Description
struct hlist_node	i_hash	Pointers for the hash list
struct list_head	i_list	Pointers for the list that describes the inode's current state
struct list_head	i_sb_list	Pointers for the list of inodes of the superblock
struct list_head	i_dentry	The head of the list of dentry objects referencing this inode
unsigned long	i_ino	inode number
atomic_t	i_count	Usage counter
umode_t	i_mode	File type and access rights
unsigned int	i_nlink	Number of hard links
uid_t	i_uid	Owner identifier
gid_t	i_gid	Group identifier
dev_t	i_rdev	Real device identifier
loff_t	i_size	File length in bytes

Virtual File System

❑ Inode Object

Table - Fields of the inode object

Type	Field	Description
struct timespec	i_atime	Time of last file access
struct timespec	i_mtime	Time of last file write
struct timespec	i_ctime	Time of last inode change
unsigned int	i_blkbits	Block size in number of bits
unsigned long	i_blksize	Block size in bytes
unsigned long	i_version	Version number, automatically increased after each use
unsigned long	i_blocks	Number of blocks of the file
unsigned short	i_bytes	Number of bytes in the last block of the file
unsigned char	i_sock	Nonzero if file is a socket
spinlock_t	i_lock	Spin lock protecting some fields of the inode
struct semaphore	i_sem	inode semaphore
struct rw_semaphore	i_alloc_sem	Read/write semaphore protecting against race conditions in direct I/O file operations
struct inode_operations *	i_op	inode operations
struct file_operations *	i_fop	Default file operations
struct super_block *	i_sb	Pointer to superblock object
struct file_lock *	i_flock	Pointer to file lock list
struct address_space *	i_mapping	Pointer to an address_space object
struct address_space	i_data	address_space object of the file
struct dquot * []	i_dquot	inode disk quotas

Virtual File System

❑ Inode Object

Table - Fields of the inode object

Type	Field	Description
struct list_head	i_devices	Pointers for a list of inodes relative to a specific character or block device
struct pipe_inode_info *	i_pipe	Used if the file is a pipe
struct block_device *	i_bdev	Pointer to the block device driver
struct cdev *	i_cdev	Pointer to the character device driver
Int	i_cindex	Index of the device file within a group of minor numbers
_ _u32	i_generation	inode version number
unsigned long	i_dnotify_mask	Bit mask of directory notify events
struct dnotify_struct *	i_dnotify	Used for directory notifications
unsigned long	i_state	inode state flags
unsigned long	dirtied_when	Dirtying time (in ticks) of the inode
unsigned int	i_flags	Filesystem mount flags
atomic_t	i_writecount	Usage counter for writing processes
void *	i_security	Pointer to inode's security structure
void *	u.generic_ip	Pointer to private data
seqcount_t	i_size_seqcount	Sequence counter used in SMP systems to get consistent values for i_size

Virtual File System

❑ inode objects

- The inode objects are also included in a hash table named inode_hashtable.
- The hash table speeds up the search of the inode object when the kernel knows both the inode number and the address of the superblock object corresponding to the filesystem that includes the file.
- Because hashing may induce collisions, the inode object includes an i_hash field that contains a backward and a forward pointer to other inodes that hash to the same position; this field creates a doubly linked list of those inodes.
- The methods associated with an inode object are also called inode operations .
- They are described by an inode_operations structure, whose address is included in the i_op field.

Virtual File System

❑ inode Operations

Here are the inode operations in the order they appear in the inode_operations table:

create(dir, dentry, mode, nameidata)
Creates a new disk inode for a regular file associated with a dentry object in some directory.

lookup(dir, dentry, nameidata)
Searches a directory for an inode corresponding to the filename included in a dentry object.

link(old_dentry, dir, new_dentry)
Creates a new hard link that refers to the file specified by old_dentry in the directory dir; the new hard link has the name specified by new_dentry.

unlink(dir, dentry)
Removes the hard link of the file specified by a dentry object from a directory.

Virtual File System

❑ inode Operations

symlink(dir, dentry, symname)
Creates a new inode for a symbolic link associated with a dentry object in some directory.

mkdir(dir, dentry, mode)
Creates a new inode for a directory associated with a dentry object in some directory.

rmdir(dir, dentry)
Removes from a directory the subdirectory whose name is included in a dentry object.

mknod(dir, dentry, mode, rdev)
Creates a new disk inode for a special file associated with a dentry object in some directory. The mode and rdev parameters specify, respectively, the file type and the device's major and minor numbers.

rename(old_dir, old_dentry, new_dir, new_dentry)
Moves the file identified by old_entry from the old_dir directory to the new_dir one. The new filename is included in the dentry object that new_dentry points to.

Virtual File System

❑ inode Operations

readlink(dentry, buffer, buflen)
Copies into a User Mode memory area specified by buffer the file pathname corresponding to the symbolic link specified by the dentry.

follow_link(inode, nameidata)
Translates a symbolic link specified by an inode object; if the symbolic link is a relative pathname, the lookup operation starts from the directory specified in the second parameter.

put_link(dentry, nameidata)
Releases all temporary data structures allocated by the follow_link method to translate a symbolic link.

truncate(inode)
Modifies the size of the file associated with an inode. Before invoking this method, it is necessary to set the i_size field of the inode object to the required new size.

permission(inode, mask, nameidata)
Checks whether the specified access mode is allowed for the file associated with inode.

Virtual File System

❑ inode Operations

setattr(dentry, iattr)
Notifies a "change event" after touching the inode attributes.

getattr(mnt, dentry, kstat)
Used by some filesystems to read inode attributes.

setxattr(dentry, name, value, size, flags)
Sets an "extended attribute" of an inode (extended attributes are stored on disk blocks outside of any inode).

getxattr(dentry, name, buffer, size)
Gets an extended attribute of an inode.

listxattr(dentry, buffer, size)
Gets the whole list of extended attribute names.

removexattr(dentry, name)
Removes an extended attribute of an inode.

- The methods listed are available to all possible inodes and filesystem types. However, only a subset of them applies to a specific inode and filesystem; the fields corresponding to unimplemented methods are set to NULL.

Virtual File System

❑ File Objects

- A file object describes how a process interacts with a file it has opened.
- The object is created when the file is opened and consists of a file structure, whose fields are described in Table.
- Notice that file objects have no corresponding image on disk, and hence no "dirty" field is included in the file structure to specify that the file object has been modified.

Table - The fields of the file object

Type	Field	Description
struct list_head	f_list	Pointers for generic file object list
struct dentry *	f_dentry	dentry object associated with the file
struct vfsmount *	f_vfsmnt	Mounted filesystem containing the file
struct file_operations *	f_op	Pointer to file operation table
atomic_t	f_count	File object's reference counter
unsigned int	f_flags	Flags specified when opening the file
mode_t	f_mode	Process access mode
Int	f_error	Error code for network write operation
loff_t	f_pos	Current file offset (file pointer)
struct fown_struct	f_owner	Data for I/O event notification via signals
unsigned int	f_uid	User's UID

Virtual File System

❑ File Objects

Table - The fields of the file object

Type	Field	Description
unsigned int	f_gid	User group ID
struct file_ra_state	f_ra	File read-ahead state
size_t	f_maxcount	Maximum number of bytes that can be read or written with a single operation (currently set to 231-1)
unsigned long	f_version	Version number, automatically increased after each use
void *	f_security	Pointer to file object's security structure
void *	private_data	Pointer to data specific for a filesystem or a device driver
struct list_head	f_ep_links	Head of the list of event poll waiters for this file
spinlock_t	f_ep_lock	Spin lock protecting the f_ep_links list
struct address_space *	f_mapping	Pointer to file's address space object

Virtual File System

File Objects

- In “Common File Model," each filesystem includes its own set of file operations that perform such activities as reading and writing a file.
- When the kernel loads an inode into memory from disk, it stores a pointer to these file operations in a file_operations structure whose address is contained in the i_fop field of the inode object.
- When a process opens the file, the VFS initializes the f_op field of the new file object with the address stored in the inode so that further calls to file operations can use these functions.
- If necessary, the VFS may later modify the set of file operations by storing a new value in f_op.

Virtual File System

File Operations

- The following list describes the file operations in the order in which they appear in the file_operations table:

lseek(file, offset, origin)

Updates the file pointer.

read(file, buf, count, offset)

Reads count bytes from a file starting at position *offset; the value *offset (which usually corresponds to the file pointer) is then increased.

aio_read(req, buf, len, pos)

Starts an asynchronous I/O operation to read len bytes into buf from file position pos (introduced to support the io_submit( ) system call).

write(file, buf, count, offset)

Writes count bytes into a file starting at position *offset; the value *offset (which usually corresponds to the file pointer) is then increased.

aio_write(req, buf, len, pos)

Starts an asynchronous I/O operation to write len bytes from buf to file position pos.

Virtual File System

❑ File Operations

readdir(dir, dirent, filldir)

Returns the next directory entry of a directory in dirent; the filldir parameter contains the address of an auxiliary function that extracts the fields in a directory entry.

poll(file, poll_table)

Checks whether there is activity on a file and goes to sleep until something happens on it.

ioctl(inode, file, cmd, arg)

Sends a command to an underlying hardware device. This method applies only to device files.

unlocked_ioctl(file, cmd, arg)

Similar to the ioctl method, but it does not take the big kernel lock. It is expected that all device drivers and all filesystems will implement this new method instead of the ioctl method.

compat_ioctl(file, cmd, arg)

Method used to implement the ioctl() 32-bit system call by 64-bit kernels.

Virtual File System

❑ File Operations

mmap(file, vma)

Performs a memory mapping of the file into a process address space

open(inode, file)

Opens a file by creating a new file object and linking it to the corresponding inode object

flush(file)

Called when a reference to an open file is closed. The actual purpose of this method is filesystem-dependent.

release(inode, file)

Releases the file object. Called when the last reference to an open file is closedthat is, when the f_count field of the file object becomes 0.

fsync(file, dentry, flag)

Flushes the file by writing all cached data to disk.

aio_fsync(req, flag)

Starts an asynchronous I/O flush operation.

fasync(fd, file, on)

Enables or disables I/O event notification by means of signals.

Virtual File System

❑ File Operations

lock(file, cmd, file_lock)
Applies a lock to the file

readv(file, vector, count, offset)
Reads bytes from a file and puts the results in the buffers described by vector; the number of buffers is specified by count.

writev(file, vector, count, offset)
Writes bytes into a file from the buffers described by vector; the number of buffers is specified by count.

sendfile(in_file, offset, count, file_send_actor, out_file)
Transfers data from in_file to out_file (introduced to support the sendfile() system call).

sendpage(file, page, offset, size, pointer, fill)
Transfers data from file to the page cache's page; this is a low-level method used by sendfile( ) and by the networking code for sockets.

get_unmapped_area(file, addr, len, offset, flags)
Gets an unused address range to map the file.

Virtual File System

File Operations

check_flags(flags)

Method invoked by the service routine of the fcntl( ) system call to perform additional checks when setting the status flags of a file (F_SETFL command). Currently used only by the NFS network filesystem.

dir_notify(file, arg)

Method invoked by the service routine of the fcntl( ) system call when establishing a directory change notification (F_NOTIFY command). Currently used only by the Common Internet File System (CIFS) network filesystem.

flock(file, flag, lock)

Used to customize the behavior of the flock() system call. No official Linux filesystem makes use of this method.

- **The methods described are available to all possible file types. However, only a subset of them apply to a specific file type; the fields corresponding to unimplemented methods are set to NULL.**

Virtual File System

❑ dentry Objects

- In the Common File Model, the VFS considers each directory a file that contains a list of files and other directories.
- Once a directory entry is read into memory, however, it is transformed by the VFS into a dentry object based on the dentry structure, whose fields are described in Table.
- The kernel creates a dentry object for every component of a pathname that a process looks up; the dentry object associates the component to its corresponding inode.
- For example, when looking up the /tmp/test pathname, the kernel creates a dentry object for the / root directory, a second dentry object for the tmp entry of the root directory, and a third dentry object for the test entry of the /tmp directory.
- Notice that dentry objects have no corresponding image on disk, and hence no field is included in the dentry structure to specify that the object has been modified.
- Dentry objects are stored in a slab allocator cache whose descriptor is dentry_cache; dentry objects are thus created and destroyed by invoking kmem_cache_alloc( ) and kmem_cache_free( ).

Virtual File System

❑ dentry Objects

Table - The fields of the dentry object

Type	Field	Description
atomic_t	d_count	Dentry object usage counter
unsigned int	d_flags	Dentry cache flags
spinlock_t	d_lock	Spin lock protecting the dentry object
struct inode *	d_inode	Inode associated with filename
struct dentry *	d_parent	Dentry object of parent directory
struct qstr	d_name	Filename
struct list_head	d_lru	Pointers for the list of unused dentries
struct list_head	d_child	For directories, pointers for the list of directory dentries in the same parent directory
struct list_head	d_subdirs	For directories, head of the list of subdirectory dentries
struct list_head	d_alias	Pointers for the list of dentries associated with the same inode (alias)
unsigned long	d_time	Used by d_revalidate method
struct dentry_operations*	d_op	Dentry methods
struct super_block *	d_sb	Superblock object of the file
void *	d_fsdata	Filesystem-dependent data
struct rcu_head	d_rcu	The RCU descriptor used when reclaiming the dentry object
struct dcookie_struct *	d_cookie	Pointer to structure used by kernel profilers
struct hlist_node	d_hash	Pointer for list in hash table entry
Int	d_mounted	For directories, counter for the number of filesystems mounted on this dentry
unsigned char[]	d_iname	Space for short filename

Virtual File System

❑ dentry Objects

Each dentry object may be in one of four states:

Free

The dentry object contains no valid information and is not used by the VFS. The corresponding memory area is handled by the slab allocator.

Unused

The dentry object is not currently used by the kernel. The d_count usage counter of the object is 0, but the d_inode field still points to the associated inode. The dentry object contains valid information, but its contents may be discarded if necessary in order to reclaim memory.

In use

The dentry object is currently used by the kernel. The d_count usage counter is positive, and the d_inode field points to the associated inode object. The dentry object contains valid information and cannot be discarded.

Negative

The inode associated with the dentry does not exist, either because the corresponding disk inode has been deleted or because the dentry object was created by resolving a pathname of a nonexistent file. The d_inode field of the dentry object is set to NULL, but the object still remains in the dentry cache, so that further lookup operations to the same file pathname can be quickly resolved. The term "negative" is somewhat misleading, because no negative value is involved.

Virtual File System

❑ dentry Objects – Methods

- The methods associated with a dentry object are called dentry operations ; they are described by the dentry_operations structure, whose address is stored in the d_op field.
- Although some filesystems define their own dentry methods, the fields are usually NULL and the VFS replaces them with default functions.

Here are the methods, in the order they appear in the dentry_operations table:

d_revalidate(dentry, nameidata)

Determines whether the dentry object is still valid before using it for translating a file pathname. The default VFS function does nothing, although network filesystems may specify their own functions.

d_hash(dentry, name)

Creates a hash value; this function is a filesystem-specific hash function for the dentry hash table. The dentry parameter identifies the directory containing the component. The name parameter points to a structure containing both the pathname component to be looked up and the value produced by the hash function..

Virtual File System

❑ dentry Objects - Methods

d_compare(dir, name1, name2)
Compares two filenames ; name1 should belong to the directory referenced by dir. The default VFS function is a normal string match. However, each filesystem can implement this method in its own way. For instance, MS-DOS does not distinguish capital from lowercase letters.

d_delete(dentry)
Called when the last reference to a dentry object is deleted (d_count becomes 0). The default VFS function does nothing.

d_release(dentry)
Called when a dentry object is going to be freed (released to the slab allocator). The default VFS function does nothing.

d_iput(dentry, ino)
Called when a dentry object becomes "negative"that is, it loses its inode. The default VFS function invokes iput( ) to release the inode object.

Virtual File System

❑ dentry Cache

- Because reading a directory entry from disk and constructing the corresponding dentry object requires considerable time, it makes sense to keep in memory dentry objects that you've finished with but might need later.
- For instance, people often edit a file and then compile it, or edit and print it, or copy it and then edit the copy. In such cases, the same file needs to be repeatedly accessed.
- **To maximize efficiency in handling dentries, Linux uses a dentry cache, which consists of two kinds of data structures:**

➢ **A set of dentry objects in the in-use, unused, or negative state**

➢ **A hash table to derive the dentry object associated with a given filename and a given directory quickly. As usual, if the required object is not included in the dentry cache, the search function returns a null value.**

- The dentry cache also acts as a controller for an inode cache . The inodes in kernel memory that are associated with unused dentries are not discarded, because the dentry cache is still using them. Thus, the inode objects are kept in RAM and can be quickly referenced by means of the corresponding dentries

Virtual File System

❑ dentry Cache

- All the "unused" dentries are included in a doubly linked "Least Recently Used" list sorted by time of insertion.
- In other words, the dentry object that was last released is put in front of the list, so the least recently used dentry objects are always near the end of the list.
- When the dentry cache has to shrink, the kernel removes elements from the tail of this list so that the most recently used objects are preserved.
- The addresses of the first and last elements of the LRU list are stored in the next and prev fields of the dentry_unused variable of type list_head.
- The d_lru field of the dentry object contains pointers to the adjacent dentries in the list.
- Each "in use" dentry object is inserted into a doubly linked list specified by the i_dentry field of the corresponding inode object (because each inode could be associated with several hard links, a list is required).
- The d_alias field of the dentry object stores the addresses of the adjacent elements in the list. Both fields are of type struct list_head.
- An "in use" dentry object may become "negative" when the last hard link to the corresponding file is deleted.
- In this case, the dentry object is moved into the LRU list of unused dentries.
- Each time the kernel shrinks the dentry cache, negative dentries move toward the tail of the LRU list so that they are gradually freed

Virtual File System

dentry Cache

- The hash table is implemented by means of a dentry_hashtable array.
- Each element is a pointer to a list of dentries that hash to the same hash table value.
- The array's size usually depends on the amount of RAM installed in the system; the default value is 256 entries per megabyte of RAM.
- The d_hash field of the dentry object contains pointers to the adjacent elements in the list associated with a single hash value.
- The hash function produces its value from both the dentry object of the directory and the filename.
- The dcache_lock spin lock protects the dentry cache data structures against concurrent accesses in multiprocessor systems.
- The d_lookup( ) function looks in the hash table for a given parent dentry object and filename; to avoid race conditions, it makes use of a seqlock.
- The _ _d_lookup( ) function is similar, but it assumes that no race condition can happen, so it does not use the seqlock.

❑ Files Associated with a Process

- Each process has its own current working directory and its own root directory.
- These are only two examples of data that must be maintained by the kernel to represent the interactions between a process and a filesystem.
- A whole data structure of type fs_struct is used for that purpose and each process descriptor has an fs field that points to the process fs_struct structure.

Table 12-6. The fields of the fs_struct structure

Type	Field	Description
atomic_t	count	Number of processes sharing this table
rwlock_t	lock	Read/write spin lock for the table fields
Int	umask	Bit mask used when opening the file to set the file permissions
struct dentry *	root	Dentry of the root directory
struct dentry *	pwd	Dentry of the current working directory
struct dentry *	altroot	Dentry of the emulated root directory (always NULL for the 80 x 86 architecture)
struct vfsmount *	rootmnt	Mounted filesystem object of the root directory
struct vfsmount *	pwdmnt	Mounted filesystem object of the current working directory
struct vfsmount *	altrootmnt	Mounted filesystem object of the emulated root directory (always NULL for the 80 x 86 architecture)

Virtual File System

❑ Files Associated with a Process

- A second table, whose address is contained in the files field of the process descriptor, specifies which files are currently opened by the process.
- It is a files_struct structure whose fields are illustrated in Table.

Table - The fields of the files_struct structure

Type	Field	Description
atomic_t	count	Number of processes sharing this table
rwlock_t	file_lock	Read/write spin lock for the table fields
Int	max_fds	Current maximum number of file objects
Int	max_fdset	Current maximum number of file descriptors
Int	next_fd	Maximum file descriptors ever allocated plus 1
struct file **	fd	Pointer to array of file object pointers
fd_set *	close_on_exec	Pointer to file descriptors to be closed on exec( )
fd_set *	open_fds	Pointer to open file descriptors
fd_set	close_on_exec_init	Initial set of file descriptors to be closed on exec( )
fd_set	open_fds_init	Initial set of file descriptors
struct file *[]	fd_array	Initial array of file object pointers

- The fd field points to an array of pointers to file objects. The size of the array is stored in the max_fds field. Usually, fd points to the fd_array field of the files_struct structure, which includes 32 file object pointers. If the process opens more than 32 files, the kernel allocates a new, larger array of file pointers and stores its address in the fd fields; it also updates the max_fds field.

Virtual File System

❑ Files Associated with a Process

- For every file with an entry in the fd array, the array index is the file descriptor.
- Usually, the first element (index 0) of the array is associated with the standard input of the process, the second with the standard output, and the third with the standard error
- Unix processes use the file descriptor as the main file identifier.
- Notice that due to dup( ) , dup2( ) , and fcntl( ) system calls, two file descriptors may refer to the same opened file that is, two elements of the array could point to the same file object.
- Users see this all the time when they use shell constructs such as 2>&1 to redirect the standard error to the standard output.
- A process cannot use more than NR_OPEN (usually, 1, 048, 576) file descriptors.

Virtual File System

❑ Files Associated with a Process

- The kernel also enforces a dynamic bound on the maximum number of file descriptors in the signal->rlim[RLIMIT_NOFILE] structure of the process descriptor; this value is usually 1,024, but it can be raised if the process has root privileges.
- The open_fds field initially contains the address of the open_fds_init field, which is a bitmap that identifies the file descriptors of currently opened files.
- The max_fdset field stores the number of bits in the bitmap.
- Because the fd_set data structure includes 1,024 bits, there is usually no need to expand the size of the bitmap.
- However, the kernel may dynamically expand the size of the bitmap if this turns out to be necessary, much as in the case of the array of file objects.

Virtual File System

Files Associated with a Process

- The kernel provides an fget( ) function to be invoked when the kernel starts using a file object.
- This function receives as its parameter a file descriptor fd.
- It returns the address in current->files->fd[fd] (that is, the address of the corresponding file object), or NULL if no file corresponds to fd.
- In the first case, fget( ) increases the file object usage counter f_count by 1.
- The kernel also provides an fput( ) function to be invoked when a kernel control path finishes using a file object.
- This function receives as its parameter the address of a file object and decreases its usage counter, f_count.
- Moreover, if this field becomes 0, the function invokes the release method of the file operations (if defined), decreases the i_writecount field in the inode object (if the file was opened for writing), removes the file object from the superblock's list, releases the file object to the slab allocator, and decreases the usage counters of the associated dentry object and of the filesystem descriptor

Virtual File System

❑ Files Associated with a Process

- The fget_light( ) and fput_light( ) functions are faster versions of fget() and fput( )
- The kernel uses them when it can safely assume that the current process already owns the file object that is, the process has already previously increased the file object's reference counter.
- For instance, they are used by the service routines of the system calls that receive a file descriptor as an argument, because the file object's reference counter has been increased by a previous open( ) system call.

Virtual File System

❑ Filesystem Types

- The Linux kernel supports many different types of filesystems.
- There are a few special types of filesystems that play an important role in the internal design of the Linux kernel.
- Filesystem registration is the basic operation that must be performed, usually during system initialization, before using a filesystem type.
- Once a filesystem is registered, its specific functions are available to the kernel, so that type of filesystem can be mounted on the system's directory tree.

Virtual File System

Special Filesystems

- While network and disk-based filesystems enable the user to handle information stored outside the kernel, special filesystems may provide an easy way for system programs and administrators to manipulate the data structures of the kernel and to implement special features of the operating system.
- Notice that a few filesystems have no fixed mount point (keyword "any" in the table). These filesystems can be freely mounted and used by the users.
- Moreover, some other special filesystems do not have a mount point at all (keyword "none" in the table). They are not for user interaction, but the kernel can use them to easily reuse some of the VFS layer code; for instance, the pipefs special filesystem, pipes can be treated in the same way as FIFO files.

Virtual File System

Special Filesystems

- Table lists the most common special filesystems used in Linux; for each of them, the table reports its suggested mount point and a short description.

Table - Most common special filesystems

Name	Mount point	Description
bdev	none	Block devices
binfmt_misc	any	Miscellaneous executable formats
devpts	/dev/pts	Pseudoterminal support (Open Group's Unix98 standard)
eventpollfs	none	Used by the efficient event polling mechanism
futexfs	none	Used by the futex (Fast Userspace Locking) mechanism
pipefs	none	Pipes
proc	/proc	General access point to kernel data structures
rootfs	none	Provides an empty root directory for the bootstrap phase
shm	none	IPC-shared memory regions
mqueue	any	Used to implement POSIX message queues
sockfs	none	Sockets
sysfs	/sys	General access point to system data
Tmpfs	any	Temporary files (kept in RAM unless swapped)
usbfs	/proc/bus/usb	USB devices

Virtual File System

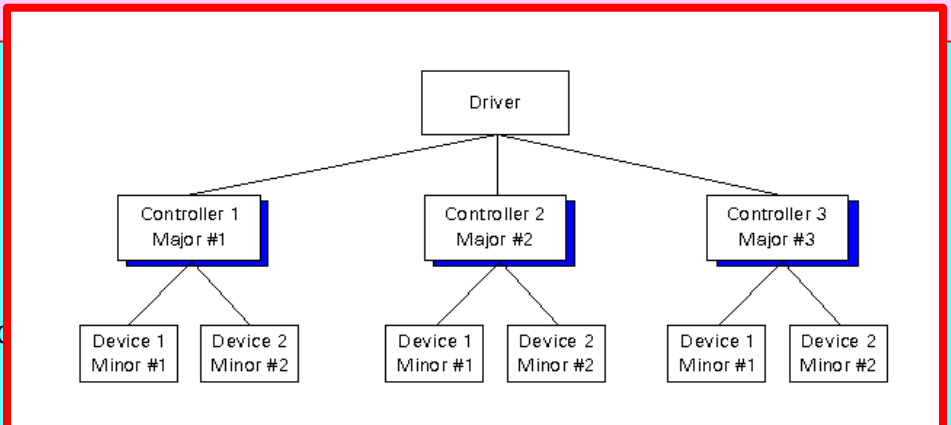
Special Filesystems

- Special filesystems are not bound to physical block devices.
- However, the kernel assigns to each mounted special filesystem a fictitious block device that has the value 0 as major number and an arbitrary value (different for each special filesystem) as a minor number.
- The set_anon_super( ) function is used to initialize superblocks of special filesystems; this function essentially gets an unused minor number dev and sets the s_dev field of the new superblock with major number 0 and minor number dev.
- Another function called kill_anon_super( ) removes the superblock of a special filesystem.
- The unnamed_dev_idr variable includes pointers to auxiliary structures that record the minor numbers currently in use.
- Although some kernel designers dislike the fictitious block device identifiers, they help the kernel to handle special filesystems and regular ones in a uniform way.

Virtual File System

❑ Major and Minor numbers of devices

- **Major and minor numbers are associated with the device special files in the /dev directory and are used by the operating system to determine the actual driver and device to be accessed by the user-level request for the special device file.**
- **The Linux kernel represents character and block devices as pairs of numbers <major>:<minor>.**
- **A given software device driver can be working with one or more hardware controllers, each of which has its own major number.**
- **Each device connected to a given controller then would have its own minor number. Thus, any single device can be identified through the major/minor number combination.**



Virtual File System

❑ Major numbers

- The major number is assigned, sequentially, to each device driver by the Installable Driver Tools (idtools) during driver installation
- Character major numbers and block major numbers are assigned separately for devices that are exclusively block or character.
- This means that two separate special files for two different device drivers may appear to have the same number assigned to them.
- A device that supports both block and character access (for example, the floppy driver), may have different major numbers for the character and block device files.

❑ Minor numbers

- Minor numbers are typically used to distinguish sub devices, but they can also be used to convey other information.
- For example, consider a floppy disk controller that can read and write data from floppies in several formats, and can also manage two floppy drives.
- When a special file associated with the floppy driver is opened, the minor number associated with the file, and available to the open( ) routine as one of its arguments, is used to identify to the floppy driver both which drive to access, and what format to assume for the I/O operation.
- In this particular case, the least significant bit of the minor number could be used to identify the drive, and the remaining bits used to indicate the format.

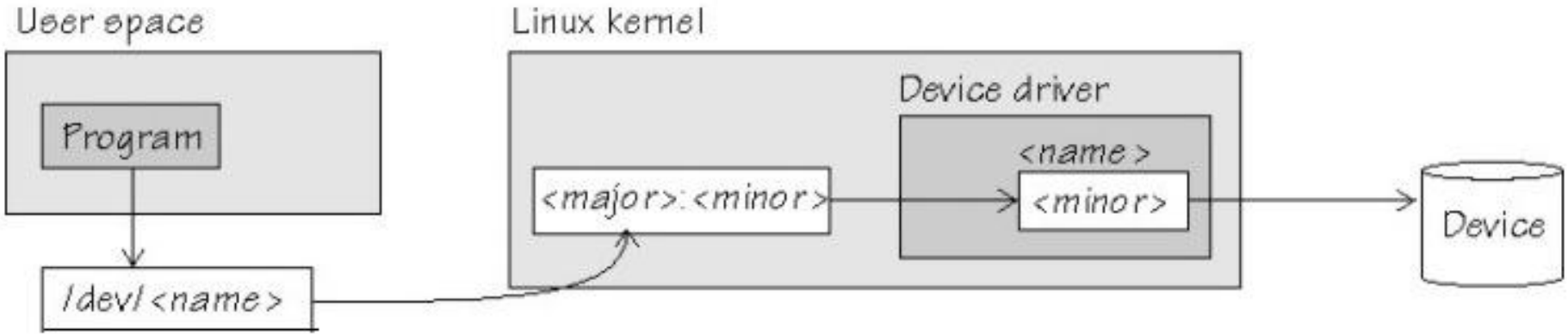
Virtual File System

❑ DIRECT ACCESS STORAGE DEVICE (DASD)

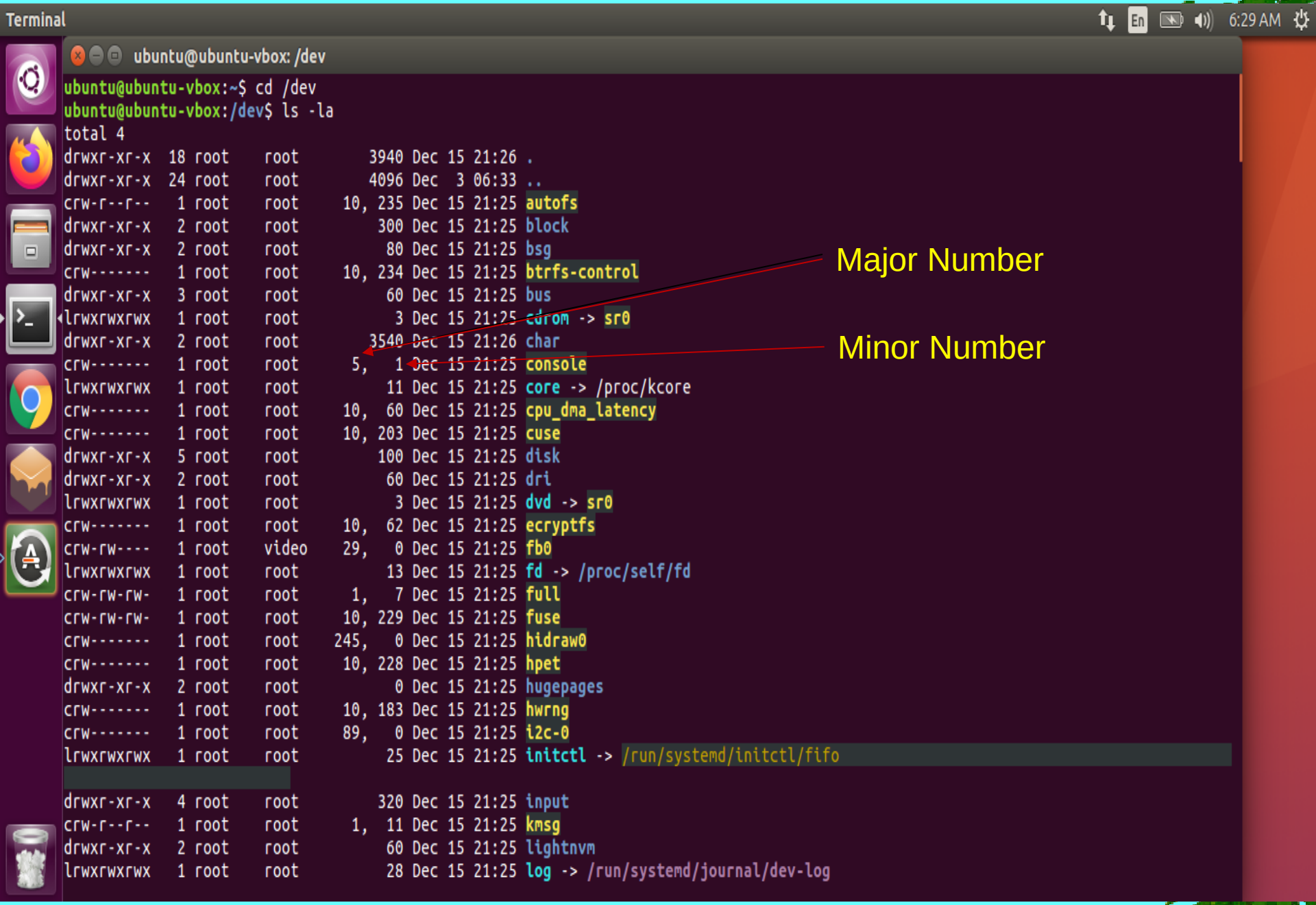
- **Direct-Access Storage Device (DASD)is a secondary storage device in which "each physical record has a discrete location and a unique address".**
- **IBM coined the term DASD as a shorthand describing hard disk drives, magnetic drums, and data cells.**
- **Later, optical disc drives and flash memory units are also classified as DASD.**
- **Some major numbers are reserved for particular device drivers. Other major numbers are dynamically assigned to a device driver when Linux boots.**
- **For example, major number 94 is always the major number for DASD devices while the device driver for channel-attached tape devices has no fixed major number.**
- **A major number can also be shared by multiple device drivers. See /proc/devices to find out how major numbers are assigned on a running Linux instance.**
- **The device driver uses the minor number <*minor*> to distinguish individual physical or logical devices.**
- **For example, the DASD device driver assigns four minor numbers to each DASD: one to the DASD as a whole and the other three for up to three partitions.**
- **Device drivers assign device names to their devices, according to a device driver-specific naming scheme.**
- **Each device name is associated with a minor number.**

Virtual File System

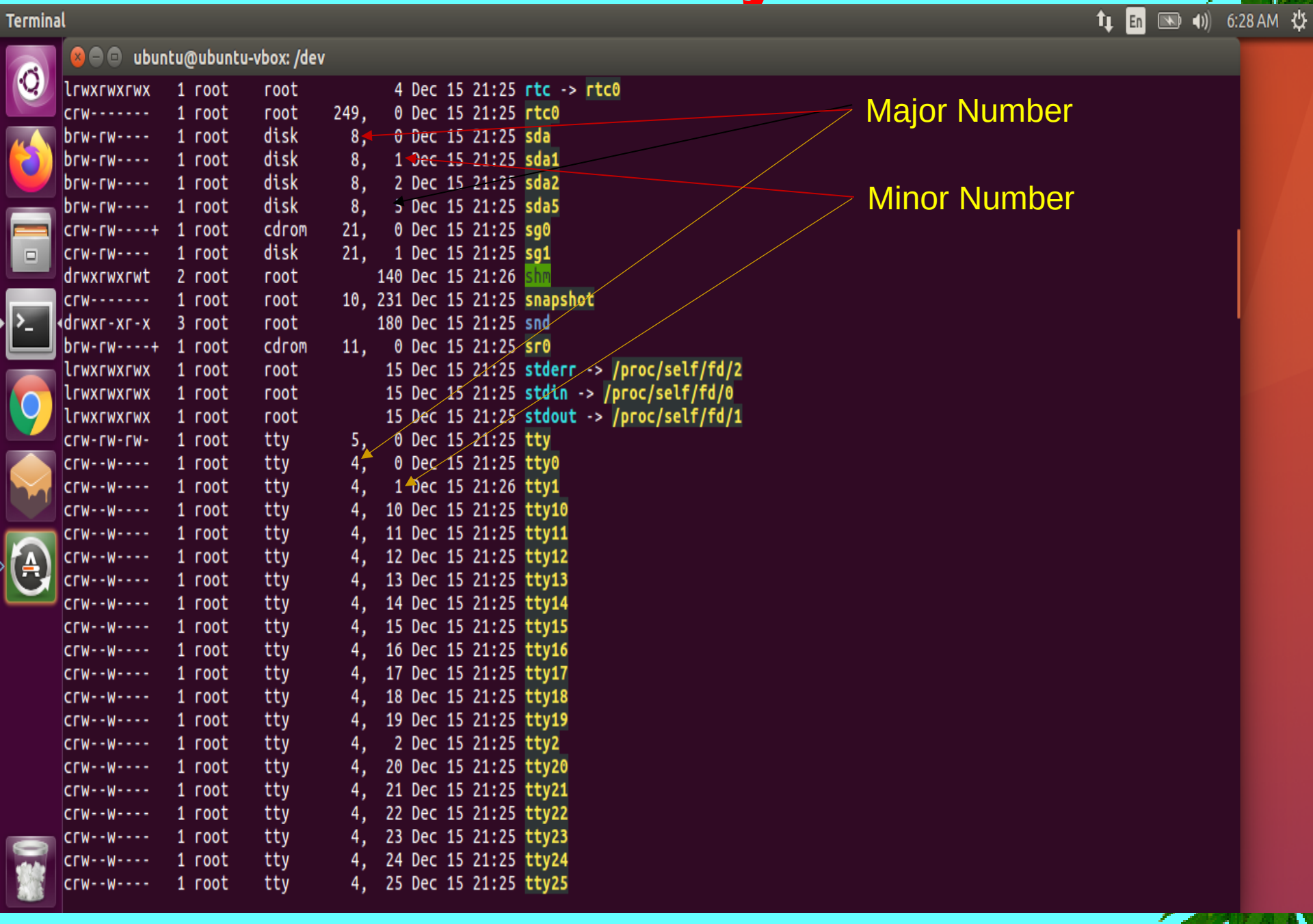
- **Major and Minor numbers are associated with the device special files in the /dev directory and are used by the operating system to determine the actual driver and device to be accessed by the user-level request for the special device file.**
- **Major number**
  - **The major number identifies the driver associated with the device type, and a minor number specifies a particular device of that type or sometimes the operation mode of that device type.**
  - **All devices controlled by the same device driver have a common major device number.**
  - **The major number is actually the offset into the kernel's device driver table, which tells the kernel what kind of device it is.**
  - **Each character and Block driver that is registered with kernel has an associated Major number.**
  - **There are total 256 [0-255] major number for character and block drivers.**
- **Minor number**
  - **The minor number is used only by the driver specified by the major number; other parts of the kernel don't use it, and merely pass it along to the driver.**
  - **It is common for a driver to control several devices; the minor number provides a way for the driver to differentiate among them.**
  - **The minor number tells the kernel special characteristics of the device to be accessed.**



Virtual File System

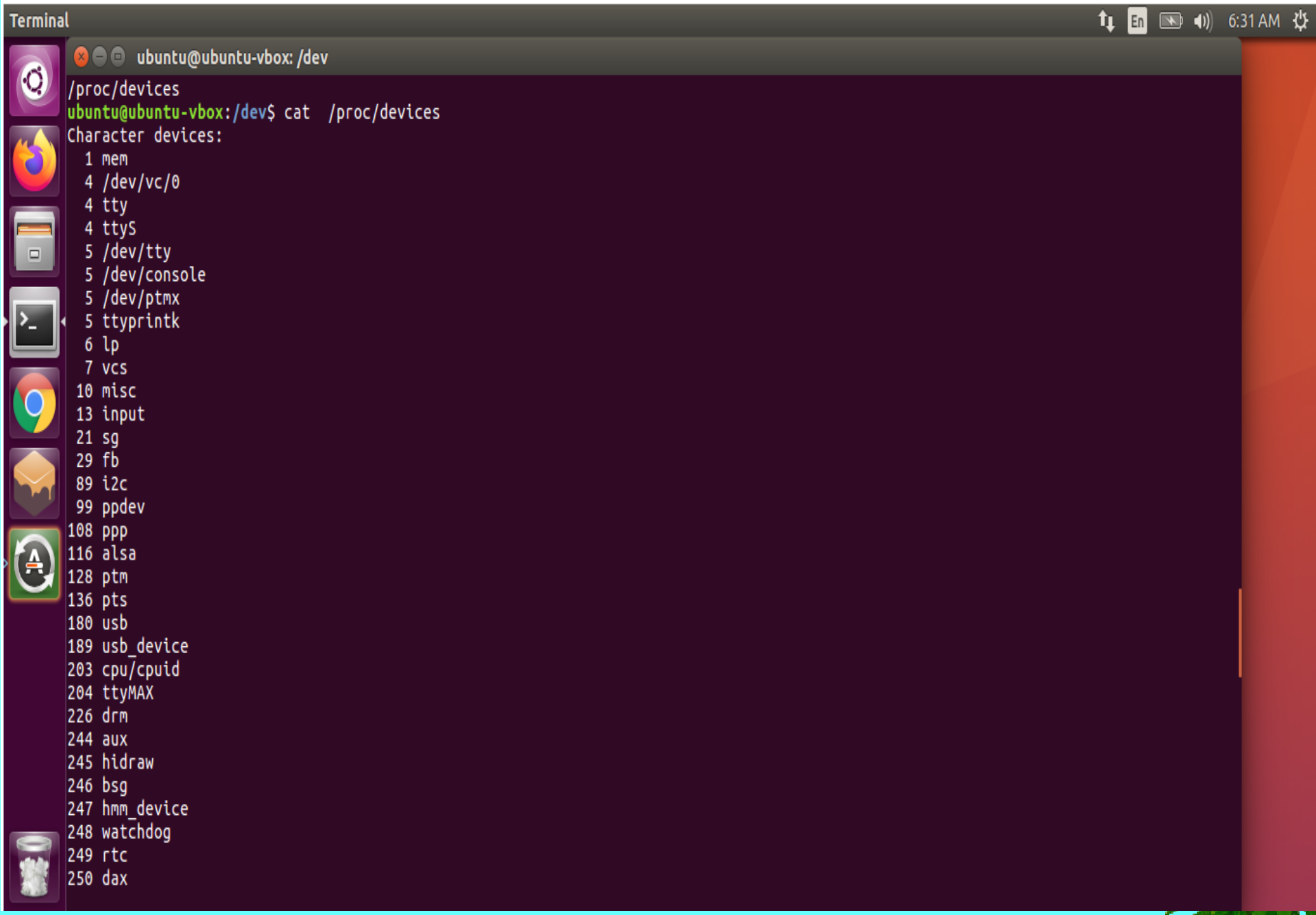


Virtual File System

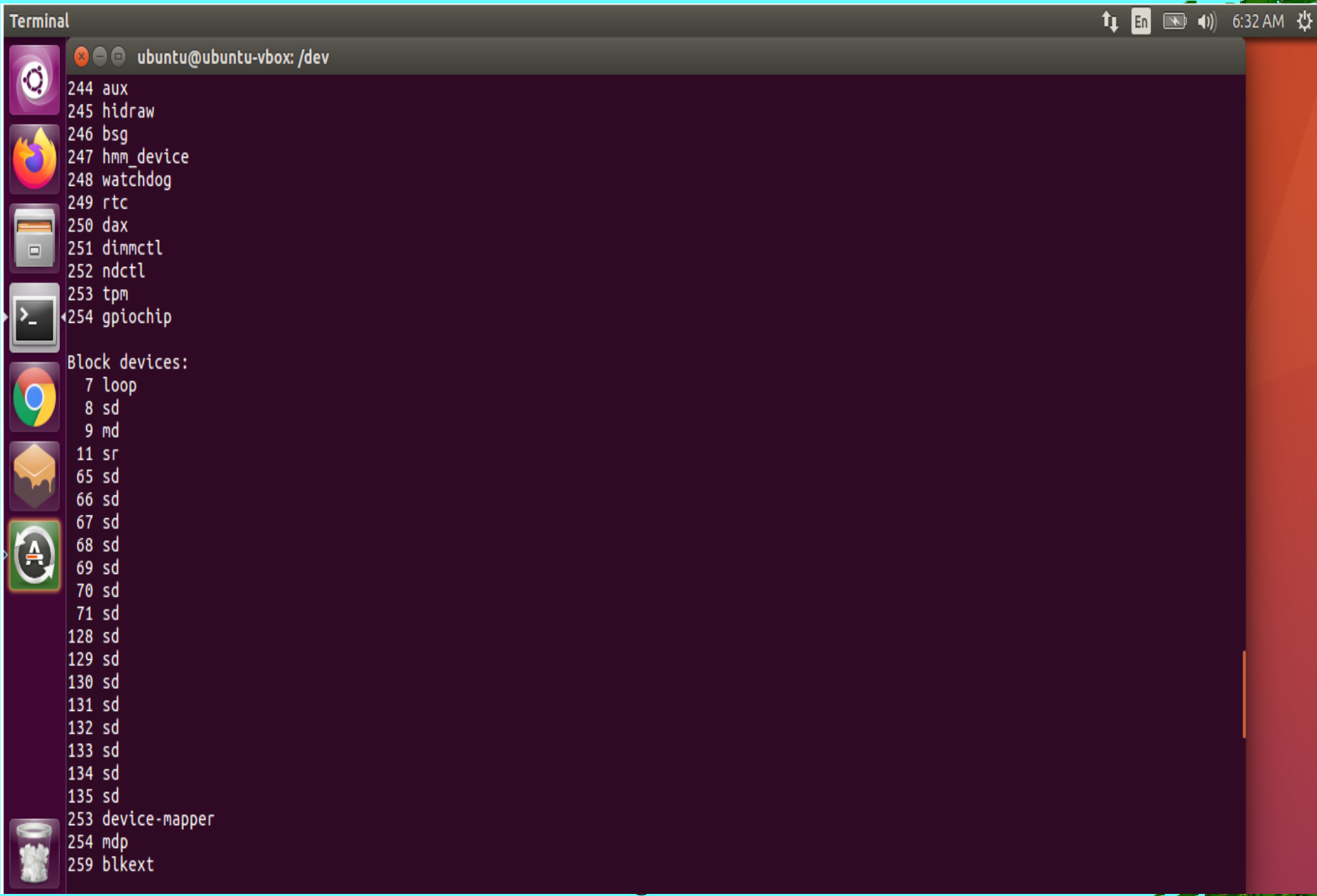



```
lrwxrwxrwx   1 root    root           4 Dec 15 21:25 rtc -> rtc0
crw-------   1 root    root    249,   0 Dec 15 21:25 rtc0
brw-rw----   1 root    disk      8,   0 Dec 15 21:25 sda
brw-rw----   1 root    disk      8,   1 Dec 15 21:25 sda1
brw-rw----   1 root    disk      8,   2 Dec 15 21:25 sda2
brw-rw----   1 root    disk      8,   5 Dec 15 21:25 sda5
crw-rw----+  1 root    cdrom    21,   0 Dec 15 21:25 sg0
crw-rw----   1 root    disk     21,   1 Dec 15 21:25 sg1
drwxrwxrwt   2 root    root         140 Dec 15 21:26 shm
crw-------   1 root    root     10, 231 Dec 15 21:25 snapshot
drwxr-xr-x   3 root    root         180 Dec 15 21:25 snd
brw-rw----+  1 root    cdrom    11,   0 Dec 15 21:25 sr0
lrwxrwxrwx   1 root    root          15 Dec 15 21:25 stderr -> /proc/self/fd/2
lrwxrwxrwx   1 root    root          15 Dec 15 21:25 stdin -> /proc/self/fd/0
lrwxrwxrwx   1 root    root          15 Dec 15 21:25 stdout -> /proc/self/fd/1
crw-rw-rw-   1 root    tty       5,   0 Dec 15 21:25 tty
crw--w----   1 root    tty       4,   0 Dec 15 21:25 tty0
crw--w----   1 root    tty       4,   1 Dec 15 21:26 tty1
crw--w----   1 root    tty       4,  10 Dec 15 21:25 tty10
crw--w----   1 root    tty       4,  11 Dec 15 21:25 tty11
crw--w----   1 root    tty       4,  12 Dec 15 21:25 tty12
crw--w----   1 root    tty       4,  13 Dec 15 21:25 tty13
crw--w----   1 root    tty       4,  14 Dec 15 21:25 tty14
crw--w----   1 root    tty       4,  15 Dec 15 21:25 tty15
crw--w----   1 root    tty       4,  16 Dec 15 21:25 tty16
crw--w----   1 root    tty       4,  17 Dec 15 21:25 tty17
crw--w----   1 root    tty       4,  18 Dec 15 21:25 tty18
crw--w----   1 root    tty       4,  19 Dec 15 21:25 tty19
crw--w----   1 root    tty       4,   2 Dec 15 21:25 tty2
crw--w----   1 root    tty       4,  20 Dec 15 21:25 tty20
crw--w----   1 root    tty       4,  21 Dec 15 21:25 tty21
crw--w----   1 root    tty       4,  22 Dec 15 21:25 tty22
crw--w----   1 root    tty       4,  23 Dec 15 21:25 tty23
crw--w----   1 root    tty       4,  24 Dec 15 21:25 tty24
crw--w----   1 root    tty       4,  25 Dec 15 21:25 tty25
```

Virtual File System



Virtual File System



Virtual File System

❑ Filesystem Type Registration

- The user configures Linux to recognize all the filesystems needed when compiling the kernel for his system.
- But the code for a filesystem actually may either be included in the kernel image or dynamically loaded as a module.
- The VFS must keep track of all filesystem types whose code is currently included in the kernel.
- It does this by performing filesystem type registration .
- Each registered filesystem is represented as a file_system_type object whose fields are illustrated in Table.

Table - The fields of the file_system_type object

Type	Field	Description
const char *	name	Filesystem name
Int	fs_flags	Filesystem type flags
struct super_block * (*)( )	get_sb	Method for reading a superblock
void (*)( )	kill_sb	Method for removing a superblock
struct module *	owner	Pointer to the module implementing the filesystem
struct file_system_type *	next	Pointer to the next element in the list of filesystem types
struct list_head	fs_supers	Head of a list of superblock objects having the same filesystem type

- All filesystem-type objects are inserted into a singly linked list. The file_systems variable points to the first item, while the next field of the structure points to the next item in the list. The file_systems_lock read/write spin lock protects the whole list against concurrent accesses.

Virtual File System

❑ Filesystem Handling

- Like every traditional Unix system, Linux makes use of a system's root filesystem : it is the filesystem that is directly mounted by the kernel during the booting phase and that holds the system initialization scripts and the most essential system programs.
- Other filesystems can be mounted either by the initialization scripts or directly by the users on directories of already mounted filesystems.
- Being a tree of directories, every filesystem has its own root directory.
- The directory on which a filesystem is mounted is called the mount point.
- A mounted filesystem is a child of the mounted filesystem to which the mount point directory belongs.
- For instance, the /proc virtual filesystem is a child of the system's root filesystem (and the system's root filesystem is the parent of /proc).
- The root directory of a mounted filesystem hides the content of the mount point directory of the parent filesystem, as well as the whole subtree of the parent filesystem below the mount point.

Virtual File System

Namespaces

- In a traditional Unix system, there is only one tree of mounted filesystems: starting from the system's root filesystem, each process can potentially access every file in a mounted filesystem by specifying the proper pathname.
- In this respect, Linux is more refined: every process might have its own tree of mounted filesystems the so-called namespace of the process.
- Usually most processes share the same namespace, which is the tree of mounted filesystems that is rooted at the system's root filesystem and that is used by the init process.
- However, a process gets a new namespace if it is created by the clone( ) system call with the CLONE_NEWNS flag set
- The new namespace is then inherited by children processes if the parent creates them without the CLONE_NEWNS flag.
- When a process mounts or unmounts a filesystem, it only modifies its namespace. Therefore, the change is visible to all processes that share the same namespace, and only to them.
- A process can even change the root filesystem of its namespace by using the Linux-specific pivot_root( ) system call.
- The namespace of a process is represented by a namespace structure pointed to by the namespace field of the process descriptor.

Virtual File System

Filesystem Mounting

- In most traditional Unix-like kernels, each filesystem can be mounted only once.
- Suppose that an Ext2 filesystem stored in the /dev/fd0 floppy disk is mounted on /flp by issuing the command:

  mount -t ext2 /dev/fd0 /flp
- Until the filesystem is unmounted by issuing a umount command, every other mount command acting on /dev/fd0 fails.
- However, Linux is different: it is possible to mount the same filesystem several times.
- Of course, if a filesystem is mounted n times, its root directory can be accessed through n mount points, one per mount operation.
- Although the same filesystem can be accessed by using different mount points, it is really unique.
- Thus, there is only one superblock object for all of them, no matter of how many times it has been mounted.
- Mounted filesystems form a hierarchy: the mount point of a filesystem might be a directory of a second filesystem, which in turn is already mounted over a third filesystem, and so on.

Virtual File System

➢ Mounting a Generic Filesystem

Actions performed by the kernel in order to mount a filesystem:

- **Consider a filesystem that is going to be mounted over a directory of an already mounted filesystem (refer to this new filesystem as "generic").**
- **The mount( ) system call is used to mount a generic filesystem; its sys_mount( ) service routine acts on the following parameters:**

- **The pathname of a device file containing the filesystem, or NULL if it is not required (for instance, when the filesystem to be mounted is network-based)**
- **The pathname of the directory on which the filesystem will be mounted (the mount point)**
- **The filesystem type, which must be the name of a registered filesystem**
- **The mount flags**
- **A pointer to a filesystem-dependent data structure (which may be NULL)**

Virtual File System

Mounting the Root Filesystem

- Mounting the root filesystem is a crucial part of system initialization.
- It is a fairly complex procedure, because the Linux kernel allows the root filesystem to be stored in many different places, such as a hard disk partition, a floppy disk, a remote filesystem shared via NFS, or even a ramdisk (a fictitious block device kept in RAM).
- To keep the description simple, assume that the root filesystem is stored in a partition of a hard disk (the most common case, after all).
- While the system boots, the kernel finds the major number of the disk that contains the root filesystem in the ROOT_DEV variable.
- The root filesystem can be specified as a device file in the /dev directory either when compiling the kernel or by passing a suitable "root" option to the initial bootstrap loader.
- Similarly, the mount flags of the root filesystem are stored in the root_mountflags variable.
- The user specifies these flags either by using the rdev external program on a compiled kernel image or by passing a suitable rootflags option to the initial bootstrap loader.

Virtual File System

➢ Mounting the Root Filesystem

Mounting the root filesystem is a two-stage procedure, shown in the following list:

1. **The kernel mounts the special rootfs filesystem, which simply provides an empty directory that serves as initial mount point.**
2. **The kernel mounts the real root filesystem over the empty directory.**

Why does the kernel bother to mount the rootfs filesystem before the real one?

- **The rootfs filesystem allows the kernel to easily change the real root filesystem.**
- **In fact, in some cases, the kernel mounts and unmounts several root filesystems, one after the other.**
- **For instance, the initial bootstrap CD of a distribution might load in RAM a kernel with a minimal set of drivers, which mounts as root a minimal filesystem stored in a ramdisk.**
- **Next, the programs in this initial root filesystem probe the hardware of the system (for instance, they determine whether the hard disk is EIDE, SCSI, or whatever), load all needed kernel modules, and remount the root filesystem from a physical block device.**

Virtual File System

- **Unmounting a Filesystem**

- **The umount( ) system call is used to unmount a filesystem.**

- **The corresponding sys_umount( ) service routine acts on two parameters:**

  - **a filename (either a mount point directory or a block device filename) and**

  - **a set of flags**

Virtual File System

❑ Pathname Lookup

- When a process must act on a file, it passes its file pathname to some VFS system call, such as open( ) , mkdir( ), rename( ) , or stat( ) .
- How the VFS performs a pathname lookup , that is, how it derives an inode from the corresponding file pathname.
- The standard procedure for performing this task consists of analyzing the pathname and breaking it into a sequence of filenames .
- All filenames except the last must identify directories.
- If the first character of the pathname is */*, the pathname is absolute, and the search starts from the directory identified by current->fs->root (the process root directory).
- Otherwise, the pathname is relative, and the search starts from the directory identified by current->fs->pwd (the process-current directory).
- Having in hand the dentry, and thus the inode, of the initial directory, the code examines the entry matching the first name to derive the corresponding inode.
- Then the directory file that has that inode is read from disk and the entry matching the second name is examined to derive the corresponding inode.
- This procedure is repeated for each name included in the path.

Virtual File System

❑ Pathname Lookup

- The dentry cache considerably speeds up the procedure, because it keeps the most recently used dentry objects in memory.
- Each such object associates a filename in a specific directory to its corresponding inode.
- In many cases, therefore, the analysis of the pathname can avoid reading the intermediate directories from disk.
- However, things are not as simple as they look, because the following Unix and VFS filesystem features must be taken into consideration:
  - The access rights of each directory must be checked to verify whether the process is allowed to read the directory's content.
  - A filename can be a symbolic link that corresponds to an arbitrary pathname; in this case, the analysis must be extended to all components of that pathname.
  - Symbolic links may induce circular references; the kernel must take this possibility into account and break endless loops when they occur.
  - A filename can be the mount point of a mounted filesystem. This situation must be detected, and the lookup operation must continue into the new filesystem.
- Pathname lookup has to be done inside the namespace of the process that issued the system call.
- The same pathname used by two processes with different namespaces may specify different files.

Virtual File System

❑ Implementations of VFS System Calls

➢ The open( ) System Call

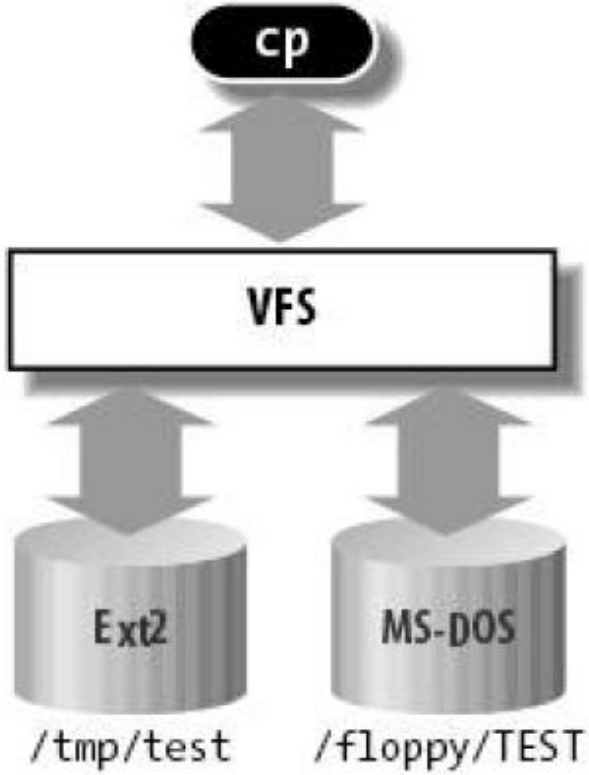
- ❖ The open() system call is serviced by the sys_open() function, which receives as its parameters the pathname filename of the file to be opened, some access mode flags flags, and a permission bit mask mode if the file must be created.
- ❖ If the system call succeeds, it returns a file descriptor that is, the index assigned to the new file in the current->files->fd array of pointers to file objects; otherwise, it returns -1.
- ❖ The open( ) system calls return two file descriptors, which are stored in the inf and outf variables.

  Then the program starts a loop: at each iteration, a portion of the/floppy/TEST file is copied into a local buffer (read( ) system call), and then the data in the local buffer is written into the /tmp/test file (write( ) system call).

Virtual File System

❑ Implementations of VFS System Calls

Figure - VFS role in a simple file copy operation



(a)

```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test",
       O_WRONLY|O_CREAT|O_TRUNC, 0600);
do {
    i = read(inf, buf, 4096);
    write(outf, buf, i);
} while (i);
close(outf);
close(inf);
```

(b)

Virtual File System

❑ Implementations of VFS System Calls

➢ The read( ) and write( ) System Calls

- The read( ) and write( ) system calls are quite similar.
- Both require three parameters: a file descriptor fd, the address buf of a memory area (the buffer containing the data to be transferred), and a number count that specifies how many bytes should be transferred.
- read( ) transfers the data from the file into the buffer, while write( ) does the opposite.
- Both system calls return either the number of bytes that were successfully transferred or -1 to signal an error condition.

Virtual File System

❑ Implementations of VFS System Calls

➢ The close( ) System Call

- ❖ **The close( ) system call receives as its parameter fd, which is the file descriptor of the file to be closed.**

- ❖ **The loop in our example code terminates when the read( ) system call returns the value 0 that is, when all bytes of /floppy/TEST have been copied into /tmp/test.**
- ❖ **The program can then close the open files, because the copy operation has completed.**

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 12 of Text Book

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010
OPERATING SYSTEM

END OF
WEEK 12 LECTURE2

VIRTUAL FILE SYSTEM (VFS)

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG